

SAFE CHARGING CHIP FOR MOBILE PHONES

An FPGA-based hardware safety layer that detects overheating, over-current and over-voltage during charging and cuts the supply before a battery fire can start

Project Synopsis

Platform: ESP32 (sensing and display) + PYNQ-Z2 / Zynq-7000 (safety decision logic)

Document status: October 2026 - prototype stage

Team / Institution: _____

Contents

1. Abstract
2. Problem Statement and Motivation
3. Objectives
4. Proposed Solution
5. System Architecture
6. Hardware Components
7. Connections and Wiring (complete)
8. Safety Logic
9. Firmware and FPGA Design
10. Implementation Roadmap and Current Status
11. Testing and Validation Plan
12. Safety Precautions
13. Applications, Limitations and Future Scope
14. Conclusion

How to read this document. Items marked **[Verified]** have been tested on the bench. Items marked **[Planned]** are the intended design and still to be built or tested. Pin numbers for the ESP32 come from the working firmware; PYNQ-Z2 pin numbers are assigned in the constraints (XDC) file during implementation.

1. Abstract

Lithium-ion batteries in mobile phones can overheat, swell and catch fire when charging goes wrong because of a faulty adapter, a damaged cable, a degraded cell or a hot environment. Most phones rely on software and a small protection circuit inside the battery pack, and these can fail without the user noticing.

This project builds an independent, hardware-level **Safe Charging Chip** that sits between the charger and the phone. It continuously measures three quantities: **voltage, current and temperature**. A decision engine classifies the charging condition as **SAFE, WARNING or FAULT**. In a FAULT it opens a relay to cut charging within milliseconds, sounds a buzzer and lights a red LED. The decision engine is implemented in **FPGA fabric on a PYNQ-Z2**, so the protective action is deterministic, runs in parallel with everything else and does not depend on an operating system or a software loop.

The ESP32 handles analog sensing and the LCD display. The ESP32 stage has already been built and checked using potentiometers that simulate the three sensors [Verified]. The next stages are the FPGA safety logic, the ESP32-to-PYNQ link and replacement of the potentiometers with the real sensors [Planned].

2. Problem Statement and Motivation

Phone battery fires are rare per charge but severe, and they usually give early warning: temperature climbs, current departs from its normal profile or the supply voltage drifts. A charging setup that watches these signals and acts quickly can prevent most incidents.

- Cheap or damaged chargers can deliver unstable voltage or excess current.
- Charging under a pillow, in a car or in direct sun raises temperature with nothing to stop it.
- Protection inside the phone is a single point of failure and the user cannot verify it.
- A separate, observable, hardware-level cut-off adds a second independent layer of protection.

3. Objectives

- Measure charging voltage, current and temperature in real time.
- Classify the condition into SAFE, WARNING and FAULT using defined thresholds.
- Cut the charging path automatically in FAULT and keep it cut until a deliberate reset.
- Give clear feedback: three LEDs, an audible alarm and a 16x2 LCD with live values.
- Implement the decision logic in an FPGA so protection is fast, parallel and deterministic.
- Demonstrate the full system safely using a dummy battery load.

4. Proposed Solution

The solution is split into two cooperating stages, each doing what it is best at.

Stage	Board	Responsibility
Sensing and display	ESP32	Read the three analog sensors, average and convert them to percentages or physical units, show values on the LCD and send them to the FPGA as a UART frame.
Safety decision and cut-off	PYNQ-Z2 (Zynq-7000 FPGA)	Receive the frame, validate it, compare against thresholds, run the SAFE/WARNING/FAULT state machine with latch and watchdog, and drive the relay, LEDs and buzzer.

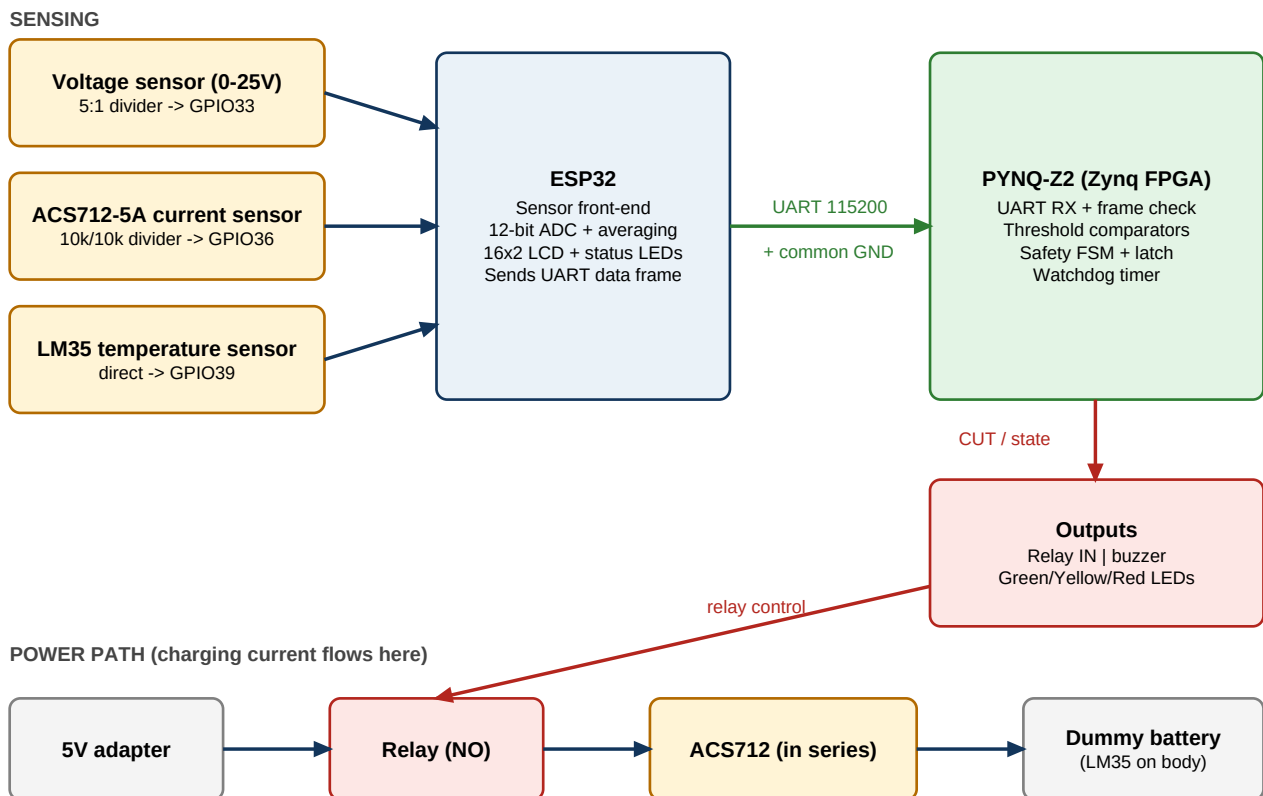
Why an FPGA?

- **Deterministic timing:** the comparison and cut-off happen in a fixed number of clock cycles, with no scheduler or interrupts.
- **Parallelism:** all three channels are checked at the same time every clock cycle.
- **Independence:** the safety path has no operating system or application code that can hang.
- **Chip story:** the Verilog design is a real digital IP block that could later be taken to a small ASIC or an inexpensive CPLD.

Design principle: the system is **fail-safe**. The relay uses the Normally-Open contact, so if power, the ESP32 or the PYNQ is lost, or the data link drops, the charging path opens and charging stops.

5. System Architecture

The diagram shows the signal flow. Sensors feed the ESP32, the ESP32 sends data to the PYNQ-Z2, and the PYNQ-Z2 controls the relay in the power path.



Working principle

- The adapter's positive line passes through the relay, then through the ACS712 current sensor, to the dummy battery. The return line goes straight back to the adapter.
- The voltage sensor measures across the battery terminals. The LM35 is fixed to the battery body.
- Every 500 ms (adjustable) the ESP32 samples all three sensors, averages the samples, converts them and transmits a frame to the PYNQ-Z2.
- The FPGA takes the highest of the three normalised values, evaluates the state and updates the outputs.
- In FAULT the relay is released, the red LED and buzzer turn on, and the state stays latched until the reset button is pressed with all readings back in the safe range.

6. Hardware Components

Component	Qty	Role in the project
PYNQ-Z2 board (Zynq-7000)	1	Runs the safety FSM in FPGA fabric; Jupyter/Python side used for logging and demo plots.
ESP32 development board	1	Analog sensing, LCD driver, UART transmitter to the PYNQ-Z2.
LM35 temperature sensor	1	Battery temperature, 10 mV per degree C output.
ACS712 5A current sensor module	1	Charging current, 185 mV per A, centred at half of supply.
0-25V voltage sensor module	1	Charging voltage via an on-board 5:1 divider.
5V 1-channel relay module	1	Opens and closes the charging path. Active-HIGH in this build.
Buzzer	1	Audible alarm in FAULT.
LEDs (green, yellow, red)	3+	Status: SAFE, WARNING, FAULT. Each uses a 220 ohm series resistor.
16x2 LCD with I2C backpack (0x27)	1	Live V, C, T and status.
3x 10k potentiometers	3	Stand-ins for the sensors during firmware development [Verified].
3-channel level converter (x3)	3	Optional 5V/3.3V signal conditioning where needed.
Dummy battery and 5V adapter	1 each	Safe test load and source. No real phone is used for fault tests.
1N4007 diodes	2	Reverse-polarity protection on the input and flyback protection for a bare relay coil.
100 nF capacitors	5	Decoupling at sensors and RC filtering on ADC inputs.
10 uF capacitors	2	Bulk decoupling near the ESP32 and relay supply.
220 ohm resistors	10	LED current limiting.
10k resistors	5	ACS712 output divider, pull-downs for the push buttons.
Push buttons	2+	Fault reset (acknowledge) and spare/mode.
2x 1.5V batteries	2	Low-energy test source for early bench tests.

7. Connections and Wiring (complete)

7.1 ESP32 pin map [Verified in firmware]

Function	ESP32 pin	Connected to	Notes
Voltage input	GPIO33 (ADC1)	Voltage sensor OUT (potentiometer in simulation)	12-bit ADC, 0-4095 maps to 0-100%
Current input	GPIO36 (VP, ADC1)	ACS712 OUT through divider (potentiometer in simulation)	Input only pin
Temperature input	GPIO39 (VN, ADC1)	LM35 OUT (potentiometer in simulation)	Input only pin
Green LED	GPIO2	LED anode via 220 ohm, cathode to GND	HIGH = on
Yellow LED	GPIO4	LED anode via 220 ohm, cathode to GND	HIGH = on
Red LED	GPIO15	LED anode via 220 ohm, cathode to GND	HIGH = on
Buzzer	GPIO25	Buzzer + (through driver if current is high), - to GND	HIGH = on
Relay	GPIO26	Relay module IN. VCC to 5V, GND to GND	HIGH = relay on (charging allowed)
LCD SDA	GPIO21	LCD I2C backpack SDA	Address 0x27, 16x2
LCD SCL	GPIO22	LCD I2C backpack SCL	Power the backpack from 5V

GPIO2 and GPIO15 are boot-strapping pins on the ESP32. Keep the LED resistors in place and do not hold these pins externally at boot.

7.2 Sensor signal conditioning [Planned]

The ESP32 ADC accepts 0 to 3.3 V only. Each sensor must be brought into that range.

Sensor	Supply	Output behaviour	Conditioning	To ESP32
LM35	5V (needs 4V or more)	10 mV per degree C. 60 degrees C = 0.60 V	None needed. Add 100 nF to GND at the ADC pin.	GPIO39
ACS712-5A	5V	2.5 V at 0 A, plus 185 mV per A. 5 A = 3.43 V (too high)	10k + 10k divider halves the signal (1.25 V at 0 A). Software multiplies by 2 and subtracts 2.5 V.	GPIO36
Voltage module	Measured line	Vin divided by 5. A 5 V line gives 1 V	None for lines up to about 16 V. Add 100 nF at the ADC pin.	GPIO33

All sensor grounds, the ESP32 GND, the PYNQ-Z2 GND and the adapter GND must be joined at a single common point (star ground). Without a common ground the readings drift and the UART link is unreliable.

7.3 Power path (high-current side) [Planned]

From	To	Purpose
Adapter +5V	Relay COM	Source enters the relay.
Relay NO (normally open)	ACS712 IP+	Charging path is closed only when the relay is energised.
ACS712 IP-	Dummy battery +	Measures the current in series with the load.
Dummy battery -	Adapter GND	Return path (common ground point).
Voltage module VIN+ / VIN-	Across battery + / -	Measures the voltage the battery actually sees.
LM35 (thermally attached)	Battery body	Measures battery temperature.
1N4007 (series, input)	Adapter + side	Reverse-polarity protection for the whole stage.

7.4 ESP32 to PYNQ-Z2 link [Planned]

Signal	ESP32	PYNQ-Z2	Notes
Data (UART)	TX2 = GPIO17	Any 3.3V Arduino-header or Pmod IO set as UART RX in the XDC file	115200 baud, 8N1. Both boards are 3.3V, so no level shifting.
Ground	GND	GND (Arduino header or Pmod)	Mandatory common ground.
Fault reset button	-	PYNQ-Z2 onboard push button (e.g. BTN0)	Handled in the FPGA. Debounced in Verilog.

Data frame (5 bytes, sent every 500 ms)

Byte	Content	Detail
0	Header	0xA5 fixed start marker
1	Voltage	0-100 (percentage of the configured range)
2	Current	0-100
3	Temperature	0-100
4	Checksum	XOR of bytes 1, 2 and 3

7.5 PYNQ-Z2 outputs [Planned]

Output	Active level	Goes to	Notes
relay_ctrl	HIGH = charging allowed	Relay module IN	Drops LOW in FAULT, on watchdog timeout and on reset of the FPGA.
led_green / led_yellow / led_red	HIGH	LEDs via 220 ohm	Mirror the state. Onboard LEDs can be used first for bring-up.
buzzer	HIGH	Buzzer driver	On in FAULT.

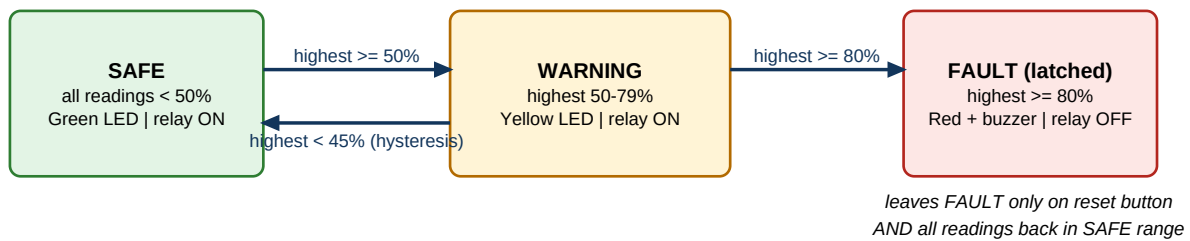
If the relay module is driven from the PYNQ-Z2, the module's own supply stays at 5V from the adapter or ESP32 board while only its IN pin connects to the 3.3V PYNQ pin. Confirm that the module triggers reliably at 3.3V. If not, use one channel of the level converter or a transistor stage.

7.6 Grounding, decoupling and protection summary

- Star ground: adapter -, ESP32 GND, PYNQ GND, sensor GND and relay GND meet at one point.
- 100 nF capacitor from each sensor output (at the ESP32 pin) to GND to reduce ADC noise.
- 10 uF capacitor across the 5V rail near the ESP32 and another near the relay module.
- 1N4007 flyback diode across a bare relay coil (cathode to the positive side). Relay modules usually include this already.
- 10k pull-down on each push button input so it never floats.
- Keep high-current wires (adapter, relay, ACS712) physically apart from the sensor signal wires.

8. Safety Logic

The decision uses the **highest** of the three normalised readings. This means any single bad channel is enough to raise the alarm.



State	Condition (highest reading)	Green	Yellow	Red	Buzzer	Relay
SAFE	below 50%	ON	OFF	OFF	OFF	ON
WARNING	50% to 79%	OFF	ON	OFF	OFF	ON
FAULT	80% or more	OFF	OFF	ON	ON	OFF (cut)

The 50% and 80% limits are the ones used in the verified ESP32 firmware. In the final system each channel is converted to physical units and the limits are set per channel.

Proposed physical thresholds (to be tuned during testing)

Quantity	Sensor range	Warning at	Fault at
Temperature	LM35, 0 to 100 degrees C	45 degrees C	60 degrees C
Current	ACS712-5A, 0 to 5 A	3.0 A	4.0 A
Voltage (5V charging)	0 to 25 V module	5.5 V	6.0 V

Added protections beyond the verified firmware [Planned]

- **Latched fault:** once in FAULT the relay stays off until the reset button is pressed and every reading is back in the safe range.
- **Hysteresis:** a state falls back only when the reading drops about 5 points below its entry threshold, which stops the output flickering at a boundary.
- **Watchdog:** if no valid frame (correct header and checksum) arrives for 1 second, the FPGA treats it as FAULT and cuts the relay.
- **Power-on state:** the relay is off at start-up until the first valid frame confirms SAFE.
- **Implausible readings:** a reading pinned at 0 or full scale for a long time (for example a broken sensor wire) is flagged as FAULT.

9. Firmware and FPGA Design

9.1 ESP32 firmware [Verified for the simulation stage]

- Reads the three analog channels (analogRead, 12-bit) and maps each to 0-100%.
- Takes the maximum of the three, selects SAFE, WARNING or FAULT and drives the LEDs, buzzer and relay accordingly.
- Prints the values to the 16x2 LCD and to the serial monitor at 115200 baud every 500 ms.
- Initial state: all LEDs and buzzer off, relay on.

Next firmware changes: replace the potentiometers with real sensor conversions, average 16 samples per reading, transmit the 5-byte frame on UART2 and keep the local LCD and LED logic as a backup display.

9.2 FPGA (Verilog) modules [Planned]

Module	Function
uart_rx	Receives 8N1 bytes at 115200 baud using a 16x oversampled bit clock.
frame_parser	Detects 0xA5, collects three data bytes, checks XOR checksum, outputs valid pulse plus V, C, T registers.
comparator	Computes the maximum of V, C and T and compares it with the warning and fault thresholds (with hysteresis).
safety_fsm	States SAFE, WARNING, FAULT_LATCHED. Handles reset and power-on state.
watchdog	1-second counter reset by each valid frame. Timeout forces FAULT.
button_debounce	Cleans the reset push button input.
output_driver	Drives relay_ctrl, three LEDs and buzzer from the FSM state.
top	Wires all modules and maps ports to PYNQ-Z2 pins in the XDC file.

9.3 Tools and flow

- Vivado: write Verilog, simulate each module with a testbench, synthesise and generate the bitstream.
- Jupyter on the PYNQ-Z2: load the bitstream with the PYNQ Overlay class, read back status registers and plot live data.
- Arduino IDE or PlatformIO for the ESP32 firmware.

10. Implementation Roadmap and Current Status

Step	Task	Status
1	ESP32 firmware with potentiometers, LCD, LEDs, buzzer and relay	Done [Verified]
2	Set up PYNQ-Z2 (SD image, network, Jupyter, onboard LED test)	Next
3	Install Vivado and board files	Planned
4	Write and simulate the Verilog safety FSM and testbench	Planned
5	Bring-up on the PYNQ-Z2 using onboard switches and LEDs as inputs and outputs	Planned
6	Connect ESP32 to PYNQ-Z2 over UART and verify the state follows the potentiometers	Planned
7	Replace potentiometers with LM35, ACS712 and voltage sensor with conditioning	Planned
8	Calibrate sensors and set final thresholds	Planned
9	Connect the relay, buzzer and LEDs to the PYNQ-Z2 outputs and test the power path with the dummy battery	Planned
10	Full-system demo, fault injection tests, documentation and pitch	Planned

11. Testing and Validation Plan

Test	Method	Expected result
Safe operation	All three inputs low	Green LED on, relay on, LCD shows SAFE
Warning	Raise one input to 50-79%	Yellow LED on, relay still on
Fault by each channel	Raise voltage, then current, then temperature to 80% or more	Red LED, buzzer, relay off within 100 ms of the frame
Latch	Lower input back to safe after a fault	Relay stays off until the reset button is pressed
Hysteresis	Hold a reading right at 50%	No LED flicker
Watchdog	Disconnect the UART wire	FAULT and relay off within 1 second
Corrupted frame	Send a wrong checksum	Frame ignored, no state change
Power cycle	Switch the system off and on	Relay stays off until the first valid SAFE frame
Sensor accuracy	Compare LM35, ACS712 and voltage readings with a multimeter and a reference thermometer	Within about 5% after calibration
Real heat test	Warm the LM35 with a hair dryer or warm water bath (never a real phone)	WARNING then FAULT at the configured temperatures

12. Safety Precautions

- Use only a dummy battery or a low-capacity test cell for fault tests. Never overheat or overcharge a real lithium-ion phone battery.
- Keep to the 5V DC side. Do not connect mains voltage to any part of the circuit.
- Check polarity and the common ground before powering the adapter.
- Use short, thick wires on the high-current path and keep it separate from sensor wiring.
- Do not exceed 3.3V on any ESP32 or PYNQ-Z2 input pin.
- Have a manual power switch or unplug option that does not rely on the electronics.

13. Applications, Limitations and Future Scope

Applications

- Phone chargers, power banks and charging docks.
- Public charging stations and shared charging lockers.
- Laptops, tablets, e-bikes and other lithium-ion battery charging.
- Educational demonstrator for FPGA-based safety systems.

Limitations (stated honestly)

- This is a prototype and has not been through safety certification (such as IEC/UL testing).
- The ACS712 and LM35 are low-cost sensors with limited accuracy and response time.
- Temperature is measured at the battery surface in the demo, not inside the cell.
- A relay is slow compared with the FPGA logic. A MOSFET switch would reduce cut-off delay.
- A PYNQ-Z2 is far larger than a real in-phone chip. The Verilog design is the part that carries over.

Future scope

- Replace the relay with a low-resistance MOSFET load switch for faster, silent cut-off.
- Trend analysis: detect fast temperature rise (degrees per second), not just a fixed limit.
- Add a second temperature sensor and cross-check the two.
- Log data and stream it to a dashboard or phone app from the PYNQ-Z2 or ESP32.
- Port the Verilog design to a small low-cost FPGA or CPLD for a compact chip.
- Add current-profile learning to recognise abnormal charging curves.

14. Conclusion

The Safe Charging Chip adds an independent protective layer to phone charging using three simple measurements and a fast, deterministic hardware decision. The sensing, display and alarm logic is already working on the ESP32 in a potentiometer simulation. The remaining work is to build the FPGA safety state machine on the PYNQ-Z2, connect it to the ESP32 over UART, and replace the simulated inputs with real sensors. The result is a clear, demonstrable prototype of a hardware-level battery-fire prevention system.